2.[15] The cheese counter in a supermarket is continuously mobbed by hungry customers. There are two sorts of customer: bold customers who push their way to the front of the mob and demand services; and meek customers who wait patiently for service. Request for service is denoted by the action *getcheese* and service completion is signalled by the action *cheese*.

(a)[5] Assuming that there is always cheese available, model the system with FSP for a fixed population of two bold customers and two meek customers.

(b)[5] Assuming that there is always cheese available, model the system with Petri nets (any kind).

(c)[5] For the FSP model, show that meek customers may never be served when their requests to get cheese have lower priority than those of bold customers.

Q2.a
This is A3 Q3.

a.[5] There are many simple solutions. The one is shown below
set Bold = {bold[1..2]}
set Meek = {meek[1..2]}
set Customers = {Bold,Meek}

CUSTOMER = (getcheese->CUSTOMER).
COUNTER = (getcheese->COUNTER).

||CHEESE_COUNTER = (Customers:CUSTOMER || Customers::COUNTER)

Another solution:

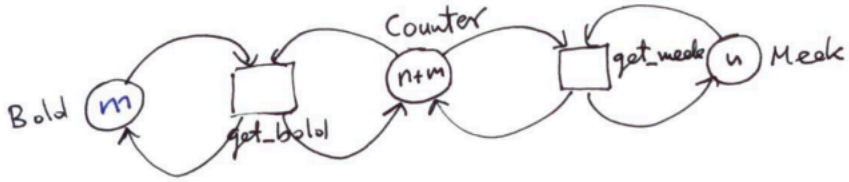
```
/* there is always cheese available */
CHEESE_DISPENSER = ( serve → cheese → CHEESE_DISPENSER ).

/* a customer is always trying to get cheese */
CUSTOMER = ( get_cheese → CUSTOMER ).

/* the hungry mob:
* The idea is to separate bold customers from
* meek customers by drawing a line, and then,
* prioritizing the actions of bold customers. */

const Customers = 4 /* maximum number of customers */

range Bold = 1..Customers/2
range Meek = Customers/2+1..Customers


||HUNGRY_MOB = (
  forall[b : Bold] bold[b]:CUSTOMER
  || forall[m : Meek] meek[m]:CUSTOMER )
```

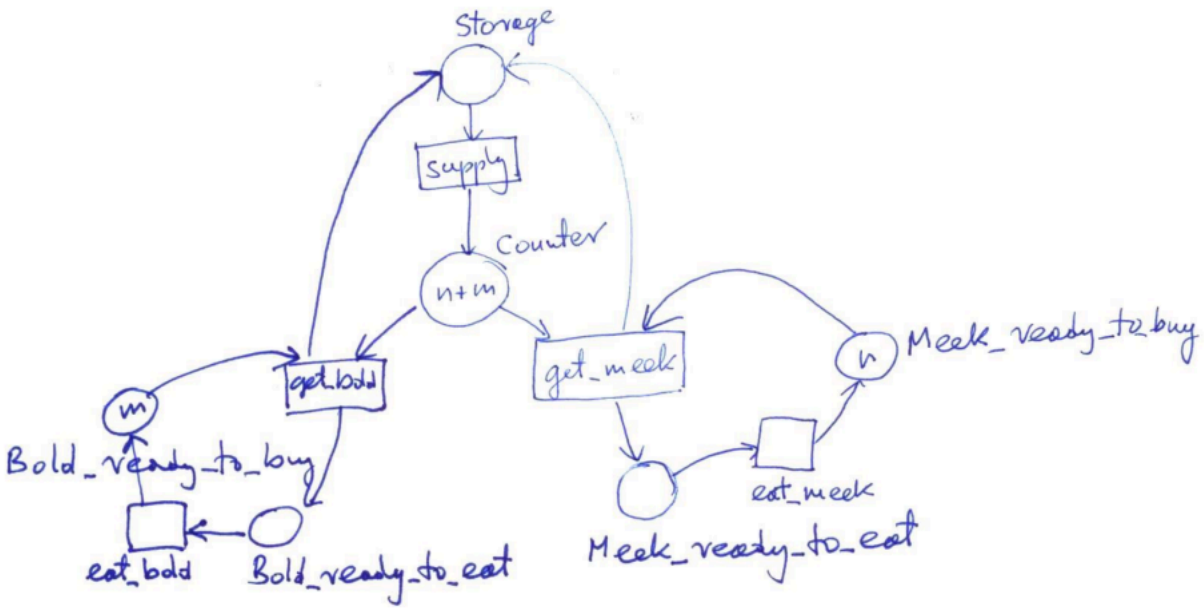


```
/* cheese counter */
||CHEESE_COUNTER = ( HUNGRY_MOB || CHEESE_DISPENSER )
  /{ {bold[Bold],meek[Meek]}.get_cheese/serve }.
```

Q2.b

b.[5] Two simple solutions in terms of Place/Transition nets.





Q2.c

c.[5] Adding priorities results in:

||CHEESE_COUNTER =

(Customers:CUSTOMER || Customers::COUNTER)>>{Meek.getcheese}.

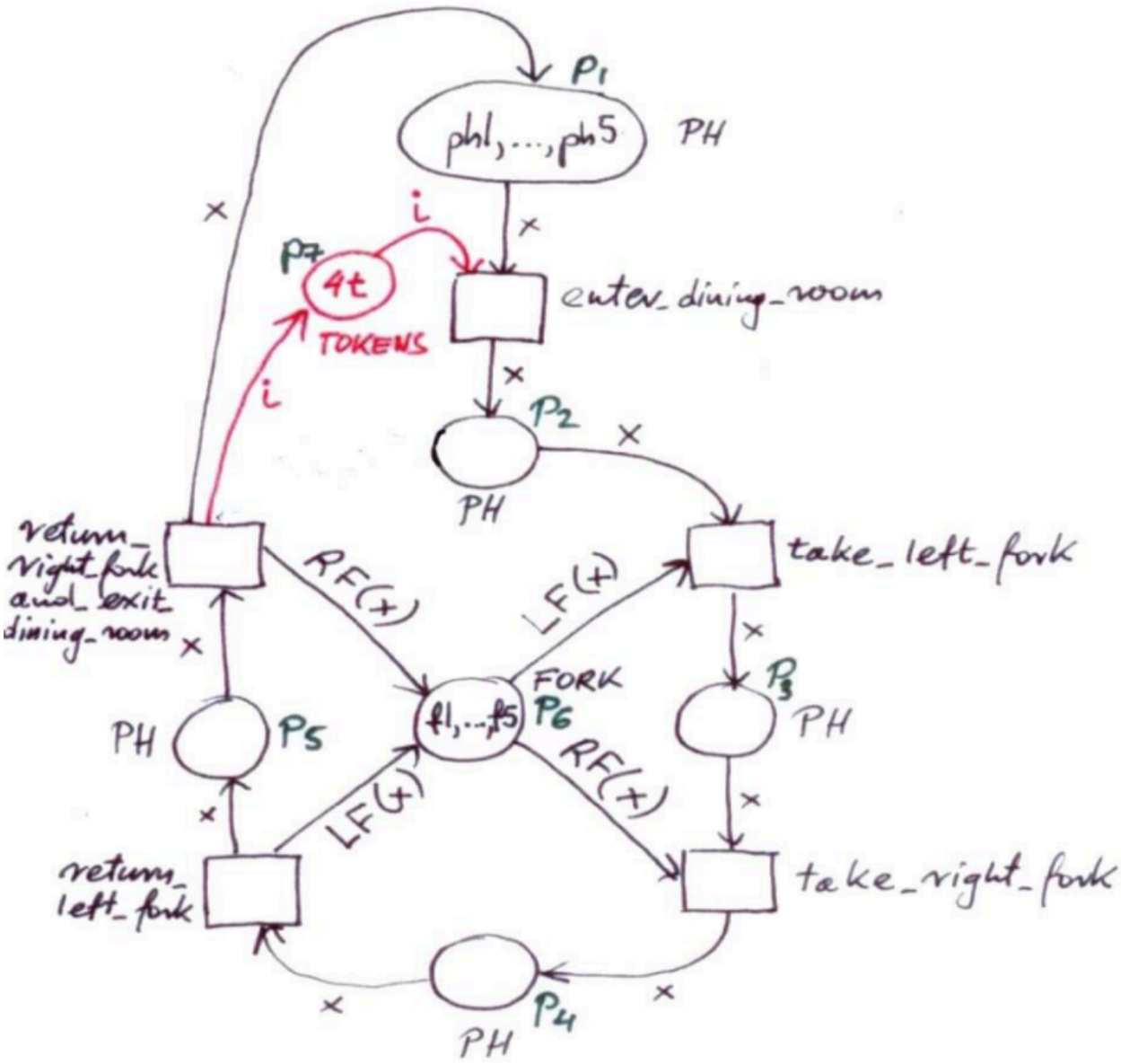
If we use

progress BOLD = {Bold.getcheese}

progress MEEK = {Meek.getcheese}

clearly Mees.getcheese get starved, as Bold.getcheese will always get executed. The same with the second solution. A choice between meek and bold will always be solved in favour of bold. Any argument of this kind should be accepted.

5.[10] Consider the Coloured Petri Net solution to Dining Philosophers with a butler, presented as a sample solution to Question 6 of Assignment 2. Prove that this solution is deadlock-free by mimicking the proof of Proposition from page 33 of Lecture Notes 12.



Q5

A2 Q6 in the question == A3 Q1 for SE3BB4 2025 students

We need to find some appropriate invariants, for example:

[inv1] $m(p_1)+m(p_2)+m(p_3)+m(p_4)+m(p_5) = ph1+ph2+ph3+ph4+ph5$

[inv2] $|m(p_7)|+|m(p_2)| = 4$

[inv3] $LF(m(p_4))+RF(m(p_4)) + m(p_6) = f1+ f2+ f3+ f4+ f4 + f5$

Now consider two cases:

(a) $m(p_4)+m(p_5) \neq 0$. Then either return_left_fork or returen_right_fork_and_exit_dinig_room can be fired.

(b) $m(p_4)+m(p_5) = 0$. Then from invariant [inv3] we have :

$LF(m(p_3)) + m(p_6) = f1+ f2+ f3+ f4+ f4 +f5$

and from invariant [inv1]:

$m(p_1)+m(p_2)+m(p_3) = ph1+ph2+ph3+ph4+ph5$.

From the definitions of LF(x) and RF(x) We have $LF(x) \neq RF(x)$ for all x = ph1, ph2, ph3, ph4, ph5.
Hence if $m(p_3) \neq 0$ then take_right_fork can be fired.
Similarly if $m(p_2) \neq 0$ then take_left_fork can be fired.
If $m(p_1) \neq ph1+ph2+ph3+ph4+ph5$, then either $m(p_3) \neq 0$ or $m(p_2) \neq 0$.
If $m(p_1) = ph1+ph2+ph3+ph4+ph5$ then $m(p_2) = 0$, and from invariant [inv2] $|m(p_7)|=4$, so enter_dining_room can be fired.

7.[15] For 'The Dining Philosophers Problem', simultaneous picking up of both forks is an abstraction of a general rule that 'the act of picking up both forks is atomic'. In other words, an order 'pick right', 'pick left' is arbitrary but once the process of picking starts, it cannot be interrupted. In practice quite often *conceptual simultaneity* is implemented as *atomicity*.
Provide a solution with FSP for Dining Philosophers with 'atomic act of (sequential) picking up both forks'.

Q7

```
FORK = ( reserve_right -> take_right -> put_right -> FORK
       | reserve_left -> take_left -> put_left -> FORK )

PHIL = (think -> reserve_forks -> USE_FORKS)
USE_FORKS = ( take_right -> take_left -> eat -> PUT_FORKS
            | take_left -> take_right -> eat -> PUT_FORKS )
PUT_FORKS = ( put_left -> put_right -> PHIL
            | put_right -> put_left -> PHIL )


||DINERS(N=5) = ( forall[i:1..N]
  ( phil[i]:PHIL || {phil[i].right,phil[(i+1)%N].left}::FORK) )
/{
 reserve_forks.1/reserve_right.1,reserve_forks.1/reserve_left.2,
  reserve_forks.2/reserve_right.2,reserve_forks.2/reserve_left.3,
 reserve_forks.3/reserve_right.3,reserve_forks.3/reserve_left.4,
  reserve_forks.4/reserve_right.4,reserve_forks.4/reserve_left.5,
 reserve_forks.5/reserve_right.5,reserve_forks.5/reserve_left.1}
```

This solution above is from A2 Q4 posted on the course webpage. So, it is guaranteed to be 100% correct.